

Name: Hasnain Saif

Sap id: 57029

Course: Data Base

Section: CS (4-1)

Project Report

INTRODUCTION

Online Food Ordering System is a database-based project that allows customers to order food from restaurants. It manages customers, restaurants, food items, orders, order details, and payments efficiently using relational database design.

3. ERD DESCRIPTION

Entities:

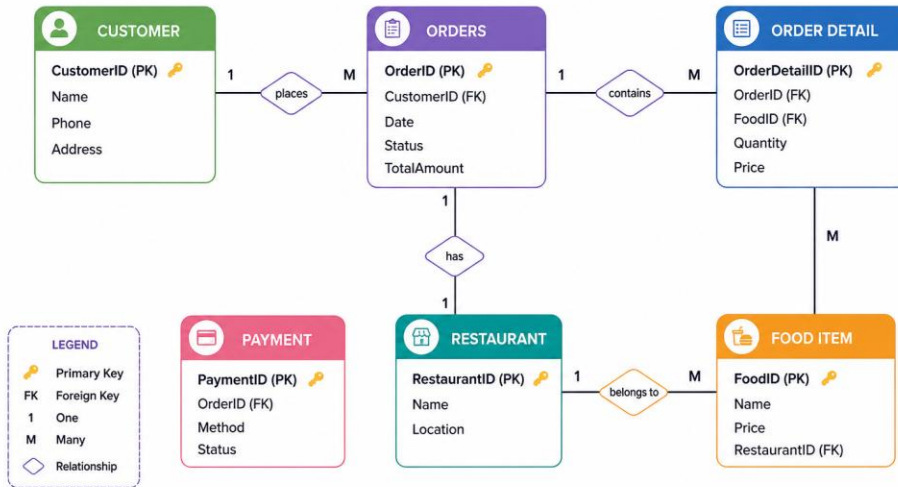
- Customer
- Restaurant
- Food
- Orders
- Order_Details
- Payment

Relationships:

- Customer places Orders (1:M)
- Restaurant has Food items (1:M)
- Order contains Order Details (1:M)
- Order has Payment (1:1)

ONLINE FOOD ORDERING SYSTEM

→ ERD (Entity Relationship Diagram) ←



4. DATABASE SCHEMA (SQL)

CUSTOMER TABLE

```
CREATE TABLE CUSTOMER (  
    customer_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50) NOT NULL,  
    phone VARCHAR2(15) UNIQUE,  
    email VARCHAR2(50)  
);
```

RESTAURANT TABLE

```
CREATE TABLE RESTAURANT (  
    restaurant_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50) NOT NULL,  
    location VARCHAR2(100)  
);
```

FOOD TABLE

```
CREATE TABLE FOOD (  
    food_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50),  
    price NUMBER CHECK (price > 0),  
    restaurant_id NUMBER,  
    FOREIGN KEY (restaurant_id) REFERENCES RESTAURANT(restaurant_id)  
);
```

ORDERS TABLE

```
CREATE TABLE ORDERS (  
    order_id NUMBER PRIMARY KEY,  
    customer_id NUMBER,  
    order_date DATE DEFAULT SYSDATE,  
    total_amount NUMBER,  
    FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id)  
);
```

ORDER DETAILS TABLE

```
CREATE TABLE ORDER_DETAILS (  
    order_detail_id NUMBER PRIMARY KEY,  
    order_id NUMBER,  
    food_id NUMBER,  
    quantity NUMBER,  
    FOREIGN KEY (order_id) REFERENCES ORDERS(order_id),  
    FOREIGN KEY (food_id) REFERENCES FOOD(food_id)  
);
```

PAYMENT TABLE

```
CREATE TABLE PAYMENT (  
    payment_id NUMBER PRIMARY KEY,  
    order_id NUMBER,  
    amount NUMBER,
```

```
method VARCHAR2(20),  
status VARCHAR2(20),  
FOREIGN KEY (order_id) REFERENCES ORDERS(order_id)  
);
```

5. SAMPLE DATA

CUSTOMER DATA

```
INSERT INTO CUSTOMER VALUES (1, 'Ali Khan', '03001234567',  
'ali@gmail.com');  
INSERT INTO CUSTOMER VALUES (2, 'Ahmed Raza', '03111234567',  
'ahmed@gmail.com');
```

RESTAURANT DATA

```
INSERT INTO RESTAURANT VALUES (1, 'Pizza Hub', 'Rawalpindi');  
INSERT INTO RESTAURANT VALUES (2, 'Burger House', 'Islamabad');
```

FOOD DATA

```
INSERT INTO FOOD VALUES (1, 'Pizza', 1200, 1);  
INSERT INTO FOOD VALUES (2, 'Burger', 500, 2);
```

ORDERS DATA

```
INSERT INTO ORDERS VALUES (1, 1, SYSDATE, 1200);  
INSERT INTO ORDERS VALUES (2, 2, SYSDATE, 500);
```

ORDER DETAILS DATA

```
INSERT INTO ORDER_DETAILS VALUES (1, 1, 1, 1);  
INSERT INTO ORDER_DETAILS VALUES (2, 2, 2, 1);
```

PAYMENT DATA

```
INSERT INTO PAYMENT VALUES (1, 1, 1200, 'Cash', 'Paid');  
INSERT INTO PAYMENT VALUES (2, 2, 500, 'Card', 'Paid');
```

6. QUERIES

◇ JOIN QUERY

```
SELECT c.name, o.order_id, o.total_amount  
FROM CUSTOMER c  
JOIN ORDERS o ON c.customer_id = o.customer_id;
```

◇ TOTAL SALES PER RESTAURANT

```
SELECT r.name, SUM(o.total_amount) AS total_sales  
FROM RESTAURANT r  
JOIN FOOD f ON r.restaurant_id = f.restaurant_id  
JOIN ORDER_DETAILS od ON f.food_id = od.food_id  
JOIN ORDERS o ON od.order_id = o.order_id  
GROUP BY r.name;
```

◇ MOST ORDERED FOOD

```
SELECT f.name, COUNT(*) AS total_orders  
FROM FOOD f  
JOIN ORDER_DETAILS od ON f.food_id = od.food_id  
GROUP BY f.name;
```

◇ TOTAL REVENUE

```
SELECT SUM(amount) AS total_revenue FROM PAYMENT;
```

◇ MAX ORDER

```
SELECT * FROM ORDERS  
WHERE total_amount = (SELECT MAX(total_amount) FROM ORDERS);
```

7. NORMALIZATION (SHORT)

- 1NF: Atomic values (no repeating groups)
- 2NF: Full dependency on primary key
- 3NF: No transitive dependency